What a posterior over a transformer costs, and what it buys

Tage Sköldefors

August 2026

Abstract

Four uncertainty methods, one CUDA implementation of a GPT-2, one card. Bayes-by-Backprop, Vadam, a learned low-rank-plus-diagonal Gaussian posterior and a five-member deep ensemble share the same kernels, the same batches in the same order and the same optimiser constants; each method's source file holds only how it draws a weight vector and how it updates the parameters behind that draw. Cost is measured in parameter-sized buffers, VRAM and milliseconds per training step, quality through one harness reporting NLPD, expected calibration error and an aleatoric/epistemic decomposition gated by an exact algebraic identity, on 11M parameters trained for 100,000 steps of character-level text8.

The ensemble has the best predictive quality and the worst calibration, and separates in- from off-distribution input by more than twice the margin of any variational arm. Vadam holds the opposite corner: cheapest posterior, best calibration, worst predictive quality, its posterior scale left at the prior because the dataset size enters the precision as Ns against λ. Converging the sample count S *widens* the ensemble's separation advantage, since under-sampling suppresses in-distribution epistemic uncertainty more than off-distribution; a separation ratio quoted without its S cannot be read. Step times are lower bounds on one thermally limited card and the off-distribution stream is synthetic, so what is measured is the ratio between arms on one implementation, not a claim about scale.

1 Introduction

A posterior over the weights of a transformer is paid for in GPU memory and GPU time. This paper prices four of them against each other on one network, one corpus and one card.

Bayes-by-Backprop [1], Vadam [11], a learned low-rank-plus-diagonal Gaussian posterior, and a deep ensemble [14] share a single CUDA implementation of GPT-2 [20]: the same kernels, the same forward and backward pass, the same batches in the same order, the same AdamW hyperparameters, the same precision. Each method's source file contains only how it draws a weight vector and how it updates the parameters behind that draw, and everything else lives in one shared header. Any difference between two arms is the posterior.

In CUDA a posterior is a count of parameter-sized buffers and a count of kernel passes over them, and cost is reported in those units and in milliseconds per step. Quality comes from one harness shared by all four arms.

2 Related work

Weight uncertainty was priced as description length by Hinton and van Camp [7], made practical for real networks by Graves [5], and turned into Bayes-by-Backprop by Blundell et al. [1] through the reparameterisation of Kingma and Welling [13]; Wen et al. [25] reduced the cost of drawing a

perturbation per example. Khan et al. [11] reuse Adam's second moment as a posterior precision, Osawa et al. [18] scale natural-gradient variational inference to ImageNet, and Shen et al. [23] apply it at GPT-2 scale. Structured covariances fitted after training include a Kronecker-factored Laplace approximation at a trained point [21], the same approximation as a recipe [3], and a low-rank-plus-diagonal factor collected from the SGD trajectory [16], which is the covariance the third arm here learns during training instead. Deep ensembles [14] match or beat variational methods on the benchmarks of Ovadia et al. [19] at E times the training cost; MC dropout [4] is the other cheap baseline, and the network here has no dropout layers. Metrics follow Guo et al. [6] for calibration and Kendall and Gal [10] for the aleatoric and epistemic split.

3 Four posteriors over one network

The shared header holds the kernels, buffer layout, forward and backward pass, dataloader, checkpoint format and evaluation harness. Each arm's source file holds only its optimiser.

3.1 Bayes-by-Backprop

A fully factorised Gaussian posterior over every parameter. Each carries a mean μ_i and an unconstrained ρ_i, and a step draws

$$w_i = \mu_i + \operatorname{softplus}(\rho_i)\,\varepsilon_i, \qquad \varepsilon \sim \mathcal{N}(0, I), \tag{1}$$

one draw shared by the whole batch, reparameterised so the gradient reaches both moments [13, 1]. The softplus keeps the standard deviation positive under AdamW's scale-free step at an initialisation near 10^{-3}. The prior is a zero-mean Gaussian of standard deviation 0.1 with a closed-form KL gradient. The objective is the ELBO, $\mathbb{E}_q[\mathrm{NLL}] + \mathrm{KL}(q\|p)/N$, with N the training token count read off the loader rather than a tuned weight; a stale value from a different corpus cost about a nat of NLL.

This is the largest fixed footprint of the four: ten parameter-sized buffers against four for a deterministic model (μ, ρ, an AdamW moment pair for each, the sampled weights and their gradient, and an accumulator per moment), and two AdamW passes per step. Regenerating ε from the index removes an eleventh buffer without changing the arithmetic, and the one-batch overfit test reaches the same loss either way.

3.2 Vadam

Adam's second moment already estimates a curvature, so Khan et al. [11] reuse it as the posterior precision. The posterior is a diagonal Gaussian whose mean alone is free, and the scale is read off the optimiser,

$$\sigma_i = \frac{1}{\sqrt{N s_i + \lambda}}, \qquad w_i = \mu_i + \sigma_i \varepsilon_i, \tag{2}$$

with s the second moment, N the training token count and λ the prior precision. Figure 1 of Khan et al. [11], transcribed unchanged:

$$m \leftarrow \gamma_1 m + (1-\gamma_1)\left(g + \tfrac{\lambda}{N}\mu\right), \tag{3}$$

$$s \leftarrow \gamma_2 s + (1-\gamma_2)\, g \circ g, \tag{4}$$

$$\mu \leftarrow \mu - \alpha\, \hat{m} / \left(\sqrt{\hat{s}} + \tfrac{\lambda}{N}\right), \tag{5}$$

with $\hat{m}, \hat{s}$ the bias-corrected moments and g the gradient at the sampled weights. That is one Adam step with the prior folded into the gradient and λ/N in place of Adam's ϵ, and nothing beyond Adam's own (m, s) is allocated.

The square root appears in the μ denominator but not in Equation 2, where σ takes s raw. Footnote 2 of Khan et al. [11] states that it "does not affect a fixed point but might still affect the steps taken"; that asymmetry is kept.

λ is derived, not tuned: matching the initialisation to the prior gives $\lambda = 1/\sigma^2_{\text{init}} = 2500$ and a prior standard deviation of 0.02. The data's contribution to the precision is Ns against λ, so small gradients leave the posterior at the prior rather than merely slowing it (Section 6). N belongs to the training run, so evaluation on validation or test reads it from the training split.

3.3 Low-rank plus diagonal covariance

This arm replaces mean-field independence with a limited amount of correlation,

$$\Sigma = AA^\top + \operatorname{diag}(d^2), \qquad A \in \mathbb{R}^{P\times r},\ d \in \mathbb{R}^P,\ r = 8. \tag{6}$$

Σ is $P \times P$ and is never formed; sampling uses the factors,

$$w_i = \mu_i + \sum_{k<r} A_{ik} z_k + d_i \varepsilon_i, \tag{7}$$

with ε drawn per weight from the index-keyed Philox stream and one $z \sim \mathcal{N}(0, I_r)$ per weight matrix per forward pass, shared by all P of its weights. That shared draw is the coupling.

$d = \operatorname{softplus}(\rho_d)$. A is signed, so softplus cannot bound it and it takes its own learning rate, 10^{-5} against 10^{-4} for the means. The prior is $\mathcal{N}(0, I/\lambda)$ with $\lambda = 100$, matching the mean-field prior standard deviation of 0.1, and its KL keeps Σ factored through

$$\operatorname{tr}(\Sigma) = \|A\|_F^2 + \|d\|^2, \qquad \log\det(\Sigma) = \log\det(M) + \textstyle\sum_i \log d_i^2, \qquad M = I_r + A^\top D^{-1} A, \tag{8}$$

which turns a $P \times P$ determinant into an $r \times r$ one. The gradients follow from the Woodbury identity and reduce to mean-field at $A = 0$, where the diagonal's stationary point is $1/\sqrt{\lambda}$.

The posterior covers the four GEMM weight matrices per block (W_{qkv}, W_{proj}, W_{fc}, W_{fc2}), or 10,616,832 of the model's 11,061,531 parameters, each with its own A, d and z, so nothing is correlated across matrices or blocks. Sampling is a separate kernel: the network calls cuBLAS, so there is no GEMM here to fuse into. The rank is compile-time so the length-r loops unroll, and a checkpoint is evaluated at the rank it was trained at.

3.4 Deep ensemble (control)

$E = 5$ independently trained deterministic models [14], differing only in the initialisation seed. The network has no dropout and the dataloader is sequential, so initialisation is the only source of diversity, and there is no bagging. Member probabilities are averaged at inference, never logits.

This arm introduces no kernels: members train through the shared forward and backward pass and the shared AdamW, and its evaluation callback returns a pointer to member s. It is the same network with the Bayesian machinery deleted, so every buffer and kernel pass the other three add is attributable to their posterior. A Bayesian arm buys its $(S+1)$-th sample for one forward pass; this arm's S is capped at the members it paid to train.

4 Experimental setup

Model. A GPT-2 [20] of the usual pre-LayerNorm transformer shape [24]: width $C = 384$, 6 heads, 6 blocks, vocabulary $V = 27$, maximum context 1024, trained at sequence length $T = 64$ and batch $B = 4$. That is 11,061,531 parameters, of which the six transformer blocks hold about 96%. Attention uses the IO-aware formulation of Dao et al. [2], so no (B, NH, T, T) score buffer is allocated. Data format, loader and reference implementation follow llm.c [9] and nanoGPT [8].

Data. text8 [17], character-level over 27 symbols (lowercase `a`–`z` plus space), in the standard split: the first 90M tokens for training, the next 5M for validation, the last 5M for test. The loader walks the corpus sequentially and hands out $B \cdot T = 256$ contiguous tokens per step, so every arm sees the same tokens in the same order. Each arm trained for 100,000 steps, or 25.6M tokens.

Hardware and precision. One RTX 2060 Max-Q, compute capability `sm_75`, 6 GB of VRAM, fp32 throughout. The four GEMMs per block go through cuBLAS; every other kernel is hand-written, benchmarked in `dev/` and checked against the single-threaded C reference.

Optimiser and RNG. AdamW [12, 15] in every arm with the same constants: learning rate 10^{-4}, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 0, no schedule. Randomness comes from Philox 4x32-10 [22], so a perturbation is a pure function of (seed, step, weight index). The backward pass regenerates the forward pass's ε from the index and no arm stores a perturbation buffer. Evaluation draws come from a seed disjoint from every training stream.

Measuring cost. Step time is wall time around the complete step, taken with CUDA events on the device that ran it, and covers the host-to-device copy, buffer clears, weight sampling, forward, backward and parameter update. VRAM is the sum of an arm's allocations, computed from the buffer layout rather than read from a driver tool, and excludes allocator slack and context overhead.

Measuring quality. Evaluation loads a checkpoint and never trains. Each arm supplies only a callback returning draw s, and the shared harness does the rest. Per position, over S draws with predictive distributions p_s and mixture $\bar{p} = \frac{1}{S}\sum_s p_s$, it reports negative log predictive density of the mixture and of a single draw, expected calibration error over 15 bins of next-token argmax confidence [6], and the decomposition [10]

$$\underbrace{H[\bar{p}]}_{\text{total}} \;=\; \underbrace{\tfrac{1}{S}\textstyle\sum_s H[p_s]}_{\text{aleatoric}} \;+\; \underbrace{\tfrac{1}{S}\textstyle\sum_s \mathrm{KL}(p_s \,\|\, \bar{p})}_{\text{epistemic}}, \tag{9}$$

in nats, accumulated in double precision. Samples are mixed as probabilities, never as logits: a logit average is a geometric mean and carries the wrong uncertainty.

Equation 9 is an identity for any $S \geq 1$ and any sampler, and both halves of the right-hand side are computed directly rather than one from the other, so asserting it tests the accumulation and not the posterior. It is also normalisation-free: an injected unnormalised row and an injected stale sample buffer both held it to 10^{-15}. The harness asserts it at 10^{-5} for every arm and exits nonzero.

Splits and the off-distribution stream. Every reported number carries its split and its S. The off-distribution stream is uniform-random token ids from a fixed seed, evaluated over the same number of positions as the in-distribution stream. Epistemic uncertainty has to rise on it, and the harness warns when it does not.

Arm	Mode	Steps	ms/step	VRAM (GB)	NLL first	NLL last	Gate
BBB	overfit	200	16.75	0.53	3.2956	0.0172	PASS
BBB	train	100000	21.56	0.53	3.2956	1.5573	PASS
Vadam	overfit	200	17.13	0.31	3.3012	0.3684	FAIL
Vadam	train	100000	17.51	0.31	3.3012	1.6299	PASS
LowR8	overfit	200	34.17	1.84	3.2922	0.0137	PASS
LowR8	train	100000	64.56	1.84	3.2922	1.5826	PASS
Ens5	overfit	200	16.54	0.44	3.2922	0.0125	PASS
Ens5	train	100000	16.95	0.44	3.2922	1.3257	PASS

Table 1: Cost per arm on one RTX 2060 Max-Q, from `runs/results.csv`. `ms/step` is wall time around the complete step and VRAM the sum of the arm's allocations, both as defined in Section 4. The ensemble's row is *one member*; multiply by $E = 5$ for the training bill. `overfit` is the one-batch memorisation gate, `train` the 100,000-step run. Step times are lower bounds (Section 8).

5 What the posterior costs

The three posteriors price differently by an order of magnitude, and the ordering follows the buffer count rather than the arithmetic. Vadam's step lands within a millisecond of the ensemble member's: its posterior is optimiser state the run had already allocated, so it adds only the sampling kernel of Equation 2. Mean-field adds a second parameter-sized tensor, its AdamW moment pair, and a second update pass. The low-rank arm adds the factor A, which at $r = 8$ is eight tensors the size of the Bayesian parameter block, read, written and sampled from every step.

The ensemble's row is one member's, and its step is the deterministic step. Its bill is E times that step and E times the training run, so the cheapest arm per step is the most expensive arm to train, and the only one whose evaluation-time S cannot be raised without paying again.

Vadam fails the one-batch overfit gate, which drives a single batch to near-zero loss. Its injected noise does not shrink as the mean fits, so its loss floors well above every other arm's while the run itself is healthy. Section 6 gives the reason.

These are lower bounds, measured on a thermally limited card over two sessions (Section 8). A difference of a few milliseconds between two rows is not a result.

6 What the posterior buys

The ensemble has the best mixture NLPD of the four and the worst ECE, by close to a factor of two; Vadam holds the opposite corner. Mean-field and the low-rank arm sit between them, separated by less than the two splits disagree about them, so those two are not ranked. Every claim here is read off `test`, and validation agrees on each one except the middle two arms' calibration ordering, which reverses between splits.

The ensemble separates in- from off-distribution input by more than twice the margin of either variational arm (Table 4), whose epistemic terms barely move off-distribution.

A larger epistemic term is not a better posterior. Vadam's in-distribution epistemic is more than three times mean-field's while its separation ratio sits with the other variational arms, far below the ensemble's. That spread is prior noise rather than learned disagreement, for the reason below.

Arm	Split	S	NLPD (single draw)	NLPD (mixture)	ECE
BBB	val	32	1.5642	1.5582	0.0240
BBB	test	32	1.6611	1.6515	0.0169
Vadam	val	32	1.7119	1.6764	0.0168
Vadam	test	32	1.7748	1.7411	0.0136
LowR	val	32	1.5982	1.5819	0.0207
LowR	test	32	1.6806	1.6643	0.0211
Ens	val	5	1.5255	1.4708	0.0490
Ens	test	5	1.6251	1.5672	0.0379

Table 2: Predictive quality, in nats, from checkpoints at 100,000 steps over 16,384 positions. Both splits are reported and neither is preferred. "Single draw" is one posterior sample for the Bayesian arms and member 0 for the ensemble, comparable within an arm and not across them. ECE is over 15 bins of next-token argmax confidence [6].

Arm	Split	S	Total	Aleatoric	Epistemic (ID)
BBB	val	32	1.602	1.591	0.0107
BBB	test	32	1.660	1.649	0.0103
Vadam	val	32	1.705	1.668	0.0372
Vadam	test	32	1.737	1.701	0.0358
LowR	val	32	1.621	1.605	0.0165
LowR	test	32	1.673	1.657	0.0164
Ens	val	5	1.628	1.561	0.0672
Ens	test	5	1.690	1.622	0.0674

Table 3: In-distribution decomposition, in nats: the entropy of the mixture, split by Equation 9 into the mean entropy of a draw (aleatoric) and the mean disagreement between draws (epistemic). The two halves are computed independently and their sum asserted against the total at 10^{-5}; the worst residual behind any number in this paper is 4.4×10^{-15}.

Why $S = 32$. Epistemic uncertainty is a Monte Carlo estimate over S draws, biased low until S converges (Figure 1). Mean-field is flat by $S = 32$ and confirmed flat out to $S = 128$. Under-sampling flatters the variational arms: from $S = 8$ to $S = 32$ their in-distribution epistemic grows faster than their off-distribution epistemic, so their separation ratios *fall* while the ensemble stays pinned at $S = E$. Converging S made the headline gap larger, not smaller.

The low-rank arm gains a further few percent of epistemic between $S = 32$ and $S = 64$ and only then stops, so at $S = 32$ mean-field sits at its limit and the low-rank arm slightly below its own. That asymmetry favours mean-field in a head-to-head and is small against the gaps this section reads. Moving the convention to $S = 64$ would remove it and costs no retraining.

Calibration degrades with S, in one arm. Mean-field's ECE rises at every doubling of S from 2 to 128 while its mixture NLPD improves, with no retraining between the two ends. The low-rank arm does not share the pattern, which argues against a binning artifact: if the estimator or the bin count were responsible, both arms would show it. It was not chased further. Any ECE quoted for mean-field carries its S or means nothing.

Arm	S	Epistemic (OOD)	OOD/ID (val)	OOD/ID (test)
BBB	32	0.0119	1.112	1.155
Vadam	32	0.0449	1.207	1.254
LowR	32	0.0208	1.261	1.268
Ens	5	0.1733	2.579	2.571

Table 4: Epistemic uncertainty on the off-distribution stream, in nats. Reported *once* per arm rather than per split: the stream is a fixed synthetic token sequence drawn from one seed, so it is the same measurement on both splits. Only the ratio is split-dependent, through its in-distribution denominator.

Vadam's posterior is the prior, and that is structural. The arm's mean left initialisation almost completely and its training loss fell like the others'. Its posterior *scale* did not move. Equation 2 makes the data's contribution to the precision Ns, weighed against $\lambda = 2500$, so for the data to matter the per-token RMS gradient has to reach $\sqrt{\lambda/N} = 5.27 \times 10^{-3}$. The median observed value is 2.71×10^{-5}, short by a factor of 195,[1] and the minibatch bias in Khan et al.'s own Theorem 1 accounts for at most a factor of 16 of that at this batch size. The posterior sits at the prior in almost every coordinate, and the arm reports the prior's spread as epistemic uncertainty.

The kernels, the sampler and the transcription were checked before this was concluded (Section 8), and rerunning the arm reproduces it. The knob that would change it is λ, and choosing λ by anything other than the initialisation is a modelling decision this comparison does not get to make.

7 Discussion

The ensemble costs E training runs and returns the best predictive quality and the widest separation between in- and off-distribution input. The variational arms buy their posterior inside one training run and return a better-calibrated model whose epistemic term barely reacts to uniform noise. Calibration and separation measure different failures.

The low-rank arm pays memory: rank 8 costs eight covariance tensors over the Bayesian parameter block, and buys correlation within a weight matrix and none across matrices. Mean-field pays a second variational tensor and a second optimiser pass. Vadam pays almost nothing and gets almost nothing, which Section 6 traces to Ns against λ. The claim that a posterior can be nearly free [11, 18, 23] is not contradicted by that. The posterior was nearly free. It was also nearly the prior. Whether the two can be separated at this gradient scale is open. The knob is λ, which stayed at the initialisation value, because choosing it by anything else is a modelling decision and not a measurement.

The estimator that reports epistemic uncertainty is itself Monte Carlo, and under-sampling suppresses the in-distribution term more than the off-distribution one, so a small S makes a variational posterior look better at separation than it is.

Reproducibility. Everything here runs from the checkpoints in the repository. No number required a retraining run, and evaluation reproduces bit for bit. The same code is packaged as a PyTorch extension that compiles the four arms into one translation unit, implements no kernel,

[1] Read from the trained checkpoint by `dev/vadam_posterior.py`, the one measurement in this paper taken from a checkpoint rather than a logged run; the rest of the arithmetic is λ and N.

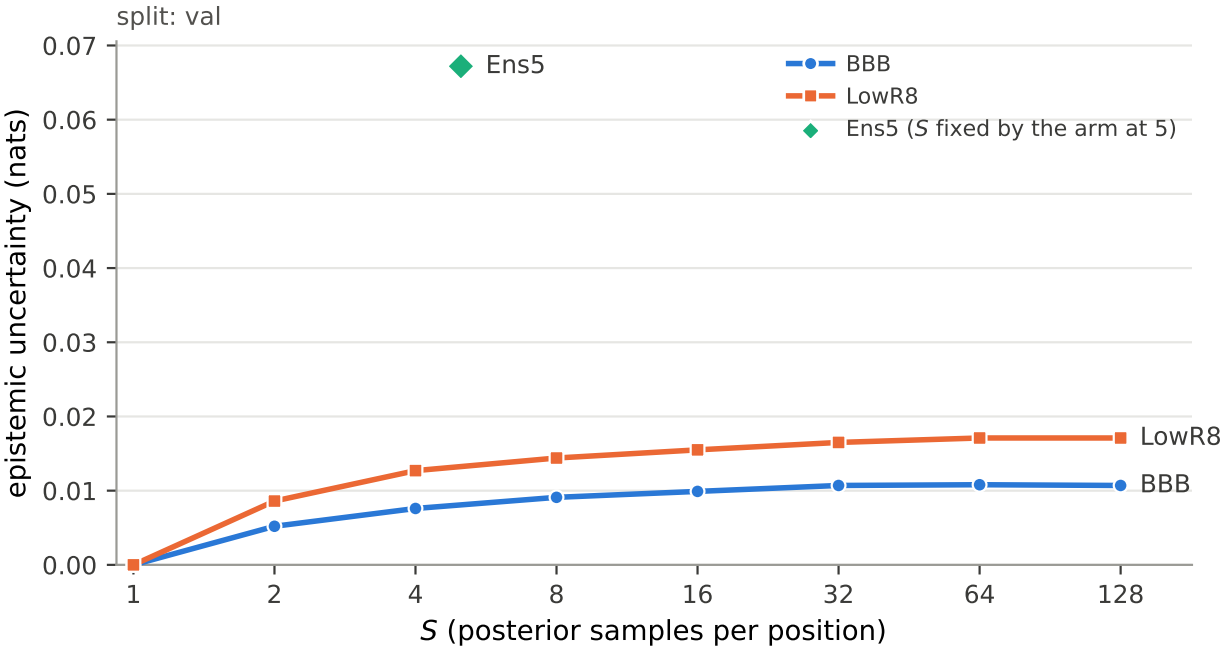


Figure 1: Epistemic uncertainty against posterior samples S, validation split, same positions as Table 3. The foot at $S = 1$ is exactly zero by construction, since one sample disagrees with nothing, and the generator refuses to plot if it is not. The ensemble is a marked point, not a curve: its S is the number of members it was paid to train.

optimiser or metric of its own, and reproduces the binaries' output exactly at a per-step overhead between −0.45% and +0.80%, inside run-to-run spread. Training does not reproduce (Section 8).

What would change the picture. A sweep over each arm's injected noise scale, which runs against saved checkpoints and would separate posterior quality from the amount of noise an arm injects. A second text domain in place of the synthetic off-distribution stream [19]. More than one seed per arm, which would turn the small gaps in these tables into resolvable ones.

8 Limitations

The scale is small. 11,061,531 parameters on a 6 GB card, picked so all four arms fit at once: a model large enough to fill the card would have priced out the low-rank arm and the five-member ensemble. What transfers is the ratio between arms on one implementation, not the absolute cost. The character-level vocabulary of 27 keeps the token embedding negligible, so no result here covers a posterior over a large embedding table. The position embedding is sized for a 1024-token context while training runs at 64, so about 3% of the parameters never see a data gradient and are held at the prior by the KL term alone, while still costing posterior state in three of the four arms.

Nothing is converged. 100,000 steps at 256 tokens per step is 25.6M tokens against a 90M token training split, 28% of one epoch. Every number is what a method does at a fixed step budget, and the ranking between arms is free to change with more steps.

Step times are lower bounds, not controlled measurements. The runs span two sessions on a card with a cooling fault. It throttled to 46–51 W at 87–89 °C against a board budget of 65–80 W, and the low-rank arm's step time drifted from 52 to 65 ms within a single run with no change in the work being done. The two sessions did not throttle alike, so the arms were not clocked under matched conditions. A gap of a few milliseconds is not a result, and what a healthy card of this class would deliver is not measured here.

$n = 1$, and the noise floor is unknown. One training run per arm, no error bars, no seed sweep. The runs are also not reproducible in principle: `encoder_backward`, `layernorm_backward` and `attention_backward` accumulate gradients with `atomicAdd`, so the order of floating-point additions varies and two identical invocations diverge. How large that divergence grows over 100,000 steps is not measured, so which differences between arms exceed it is unknown.

The off-distribution stream is synthetic. Uniform-random token ids, not shifted text. Sample disagreement has to rise where the input carries no structure, and an arm that fails that is not capturing model uncertainty. It supports no comparison with the shift literature [19], for which a second text domain would be the minimum.

Noise scale and posterior quality are confounded. Loss tracks the injected noise scale monotonically across arms, so an arm that samples less aggressively looks better on predictive metrics for a reason unrelated to its posterior. Separating the two needs a sweep over the noise scale within each arm and a comparison of curves rather than of single points, which runs against saved checkpoints.

What each code path's correctness argument covers. Each kernel was validated against the CPU reference implementation, but that reference is a single-block model and says nothing about the block loop at $L > 1$. That gap is closed by a finite-difference gradient check on a two-block configuration, run per parameter group and per block, which agrees to $\sim 10^{-4}$ relative, the same error the single-block configuration shows. Vadam's optimiser kernels carry two more checks: a double-precision host reference for the update, and a distributional test of the sampler over 20,000 draws covering the marginal variances and the absence of off-diagonal correlation. Those checks are why Section 6 attributes that arm's posterior to Ns against λ rather than to a bug. None of this covers the fused FlashAttention kernels beyond the CPU reference, and none of it is a proof.

The arms are not identical in what they cover. The low-rank arm places its posterior over the four GEMM weight matrices per block and leaves embeddings, LayerNorm parameters and biases deterministic, where the other three are Bayesian over every parameter. It covers 96% of the model rather than all of it, which flatters its cost. The ensemble evaluates at $S = E = 5$ by construction while the Bayesian arms evaluate at whatever S is requested, so a mixture metric compared across arms at different S compares different sample counts.

References

[1] Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural network. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *Proceedings of Machine Learning Research*, pages 1613–1622. PMLR, 2015. URL https://proceedings.mlr.press/v37/blundell15.html.

[2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.

[3] Erik A. Daxberger, Agustinus Kristiadi, Alexander Immer, Runa Eschenhagen, Matthias Bauer, and Philipp Hennig. Laplace redux — effortless Bayesian deep learning. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, pages 20089–20103, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/a7c9585703d275249f30a088cebba0ad-Abstract.html.

[4] Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 1050–1059. JMLR.org, 2016. URL http://proceedings.mlr.press/v48/gal16.html.

[5] Alex Graves. Practical variational inference for neural networks. In *Advances in Neural Information Processing Systems 24 (NIPS)*, pages 2348–2356, 2011. URL https://proceedings.neurips.cc/paper/2011/hash/7eb3c8be3d411e8ebfab08eba5f49632-Abstract.html.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017. URL https://proceedings.mlr.press/v70/guo17a.html.

[7] Geoffrey E. Hinton and Drew van Camp. Keeping the neural networks simple by minimizing the description length of the weights. In *Proceedings of the Sixth Annual ACM Conference on Computational Learning Theory (COLT)*, pages 5–13. ACM, 1993. doi: 10.1145/168304.168306.

[8] Andrej Karpathy. nanoGPT. https://github.com/karpathy/nanoGPT, 2022.

[9] Andrej Karpathy. llm.c: LLM training in simple, raw C/CUDA. https://github.com/karpathy/llm.c, 2024.

[10] Alex Kendall and Yarin Gal. What uncertainties do we need in Bayesian deep learning for computer vision? In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 5574–5584, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/2650d6089a6d640c5e85b2b88265dc2b-Abstract.html.

[11] Mohammad Emtiyaz Khan, Didrik Nielsen, Voot Tangkaratt, Wu Lin, Yarin Gal, and Akash Srivastava. Fast and scalable Bayesian deep learning by weight-perturbation in Adam. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 2611–2620. PMLR, 2018. URL https://proceedings.mlr.press/v80/khan18a.html.

[12] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015. URL http://arxiv.org/abs/1412.6980.

[13] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *2nd International Conference on Learning Representations (ICLR)*, 2014. URL http://arxiv.org/abs/1312.6114.

[14] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 6402–6413, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/9ef2ed4b7fd2c810847ffa5fa85bce38-Abstract.html.

[15] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *7th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2019. URL https://openreview.net/forum?id=Bkg6RiCqY7.

[16] Wesley J. Maddox, Pavel Izmailov, Timur Garipov, Dmitry P. Vetrov, and Andrew Gordon Wilson. A simple baseline for Bayesian uncertainty in deep learning. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 13132–13143, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/118921efba23fc329e6560b27861f0c2-Abstract.html.

[17] Matt Mahoney. About the test data. http://mattmahoney.net/dc/textdata.html, 2011. Large Text Compression Benchmark; source of `enwik8` and `text8`.

[18] Kazuki Osawa, Siddharth Swaroop, Mohammad Emtiyaz Khan, Anirudh Jain, Runa Eschenhagen, Richard E. Turner, and Rio Yokota. Practical deep learning with Bayesian principles. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 4289–4301, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/b53477c2821c1bf0da5d40e57b870d35-Abstract.html.

[19] Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, D. Sculley, Sebastian Nowozin, Joshua V. Dillon, Balaji Lakshminarayanan, and Jasper Snoek. Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 13969–13980, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/8558cb408c1d76621371888657d2eb1d-Abstract.html.

[20] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[21] Hippolyt Ritter, Aleksandar Botev, and David Barber. A scalable Laplace approximation for neural networks. In *6th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2018. URL https://openreview.net/forum?id=Skdvd2xAZ.

[22] John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. Parallel random numbers: As easy as 1, 2, 3. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 16:1–16:12. ACM, 2011. doi: 10.1145/2063384.2063405.

[23] Yuesong Shen, Nico Daheim, Bai Cong, Peter Nickl, Gian Maria Marconi, Clement Bazan, Rio Yokota, Iryna Gurevych, Daniel Cremers, Mohammad Emtiyaz Khan, and Thomas Möllenhoff. Variational learning is effective for large deep networks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 44665–44686. PMLR, 2024. URL https://proceedings.mlr.press/v235/shen24b.html.

[24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 5998–6008, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[25] Yeming Wen, Paul Vicol, Jimmy Ba, Dustin Tran, and Roger B. Grosse. Flipout: Efficient pseudo-independent weight perturbations on mini-batches. In *6th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2018. URL https://openreview.net/forum?id=rJNpifWAb.